Theory of Algorithms – Divide and Conquer

Patrick Marais
Department of Computer Science
University of Cape Town
patrick@cs.uct.ac.za

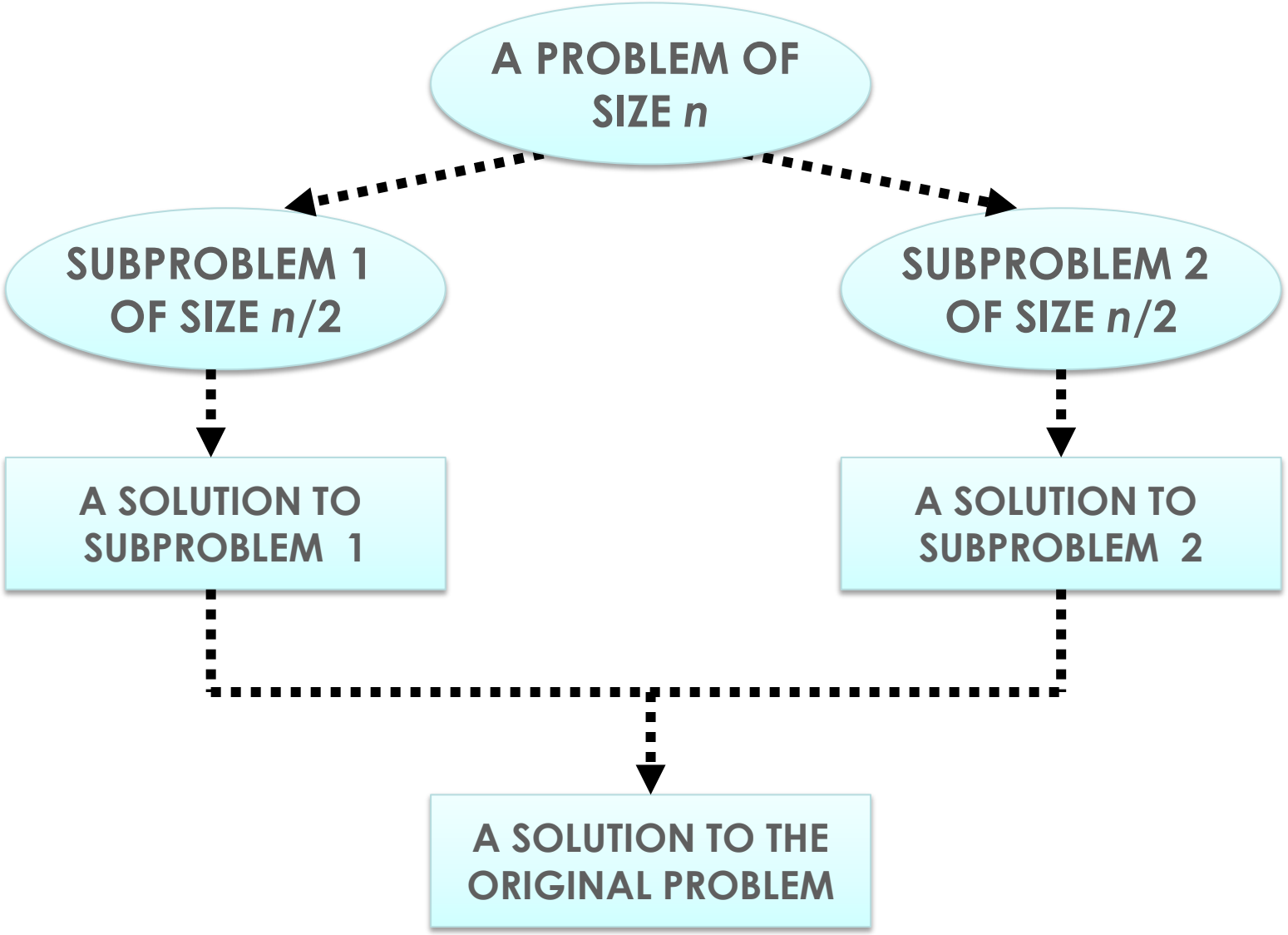




Introduction

- Best known algorithm design strategy:
  1. Divide instance of problem into two or more smaller instances
  2. Solve smaller instances recursively
  3. Obtain solution to original (larger) instance by combining these solutions
- Recurrence relations apply
  - Recurrences are of the form $T(n) = aT(n/b) + f(n)$, $a >= 1$, $b >= 2$
- Simple Example (Addition):
  - $a_0 + \ldots + a_{n-1} = (a_0 + \ldots + a_{n/2-1}) + (a_{n/2} + \ldots a_{n-1})$

Divide and Conquer Illustrated

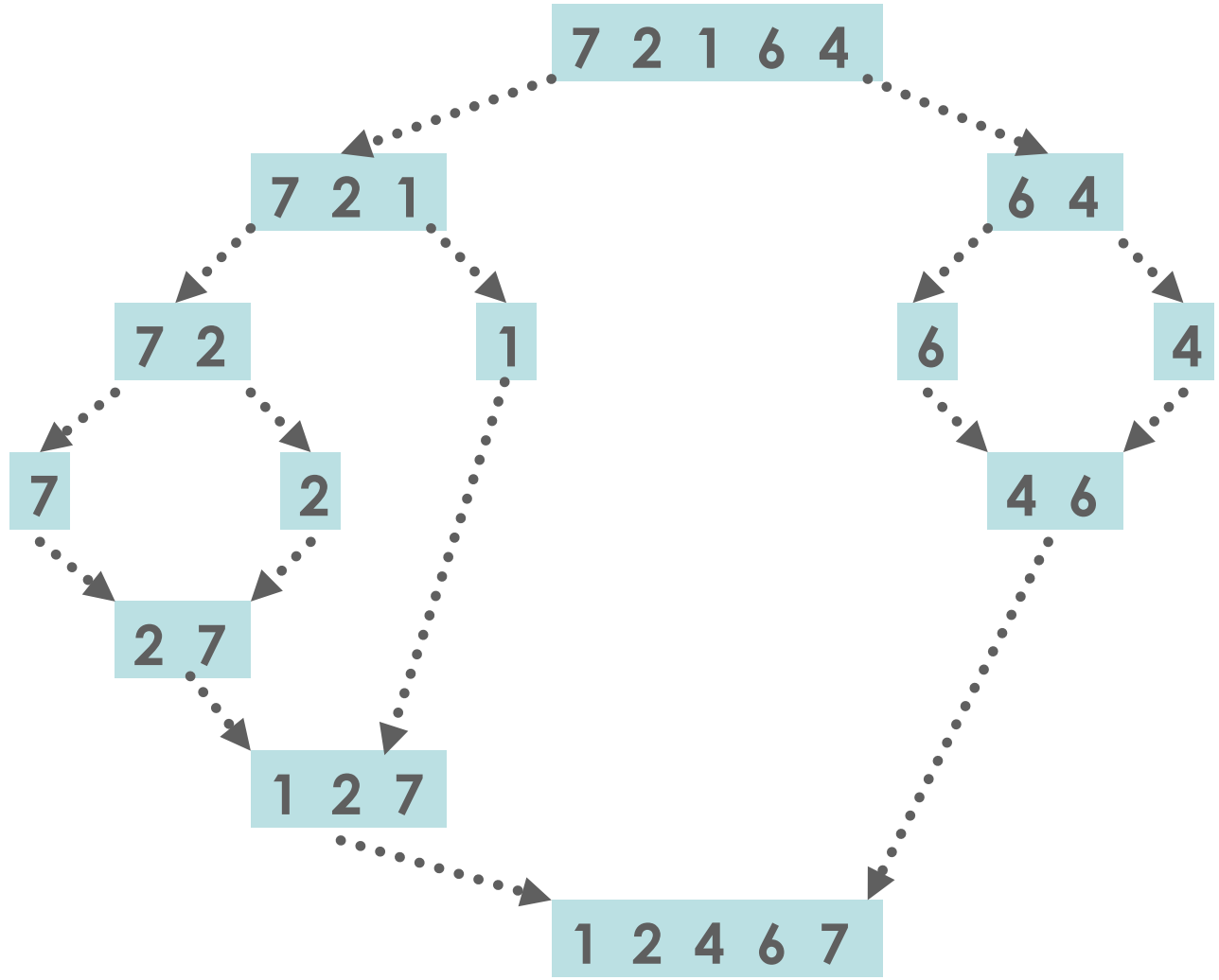


MergeSort

- Algorithm:
  1. Split *A[1..n]* in half and put copy of each half into arrays *B[1.. n/2]* and *C[1.. n/2]*
  2. Recursively MergeSort arrays B and C
  3. Merge sorted arrays B and C into array A
- Merging:
  - REPEAT until no elements remain in one of B or C
  1. Compare 1st elements in the lists for B and C
  2. Copy smaller into A, incrementing index of corresponding array
  3. Once all elements in one of B or C are processed, copy the remaining unprocessed elements from the other array into A

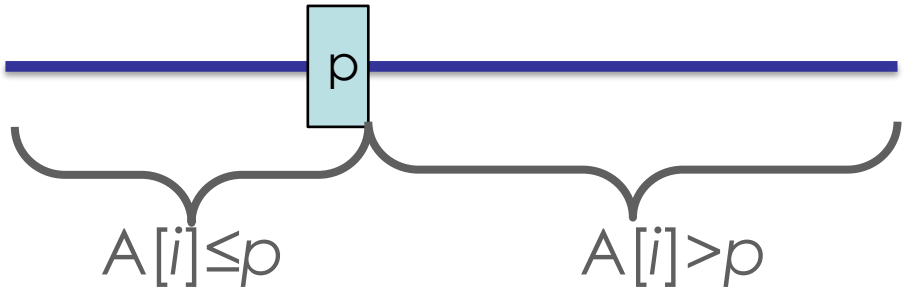
MergeSort Example

- Master Theorem:
  - $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$
  - $T(n) \in \Theta(n^d \log n)$ if $a = b^d$
- Recurrence (assume $n=2^k$):
  - $C(n) = 2\,C(n/2) + C_{merge}(n)$ *for* $n > 1$, $C(1) = 0$
  - $C_{merge}(n) = n - 1$ in the worst case
- Efficiency (all cases):
  - $a = b^d$ $(2 = 2^1)$ in Master Theorem
    $\Theta(n \log n)$
- Pro: stable (vs quicksort, heapsort)
- Con: uses extra space $\Theta(n)$



QuickSort (Reminder)

- Select a *pivot* (partitioning element) – this is A[0] usually
- Rearrange the list into two sub-lists
  - All elements positioned before the pivot are ≤ the pivot
  - Those positioned after the pivot are > the pivot
  - Requires a pivoting algorithm
- Exchange the pivot with the last element in the first sub-list
  - The pivot is now in its final position
- QuickSort the two sub-lists

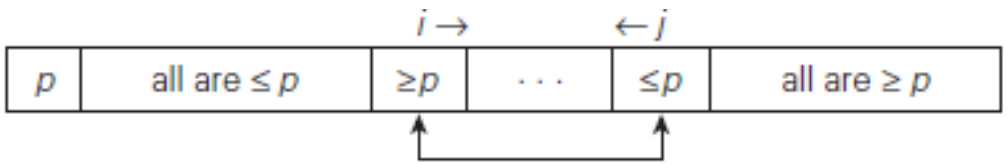


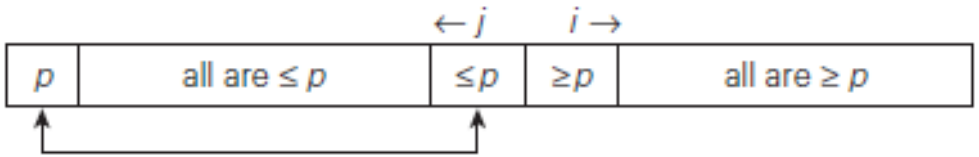
ALGORITHM *Quicksort*$(A[l..r])$

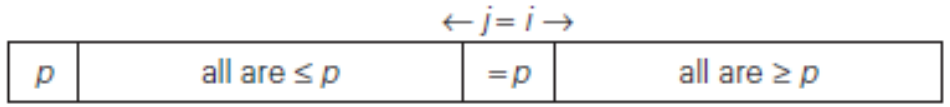
//Sorts a subarray by quicksort
//Input: Subarray of array $A[0..n-1]$, defined by its left and right
// indices l and r
//Output: Subarray $A[l..r]$ sorted in nondecreasing order
if $l < r$
  $s \leftarrow$*Partition*$(A[l..r])$ //s is a split position
  Quicksort$(A[l..s-1])$
  Quicksort$(A[s+1..r])$

ALGORITHM *HoarePartition*$(A[l..r])$

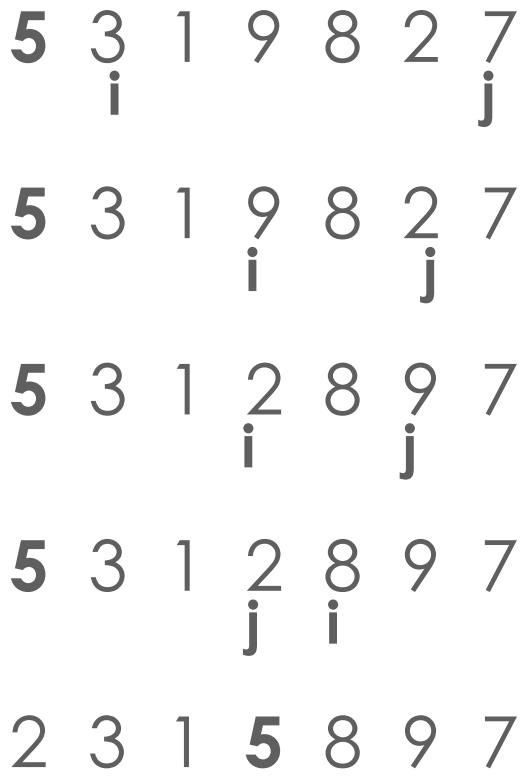
//Partitions a subarray by Hoare's algorithm, using the first element
// as a pivot
//Input: Subarray of array $A[0..n-1]$, defined by its left and right
// indices l and r $(l < r)$
//Output: Partition of $A[l..r]$, with the split position returned as
// this function's value
$p \leftarrow A[l]$
$i \leftarrow l$; $j \leftarrow r+1$
repeat
  repeat $i \leftarrow i+1$ **until** $A[i] \geq p$
  repeat $j \leftarrow j-1$ **until** $A[j] \leq p$
  swap$(A[i], A[j])$
until $i \geq j$
swap$(A[i], A[j])$ //undo last swap when $i \geq j$
swap$(A[l], A[j])$
return j



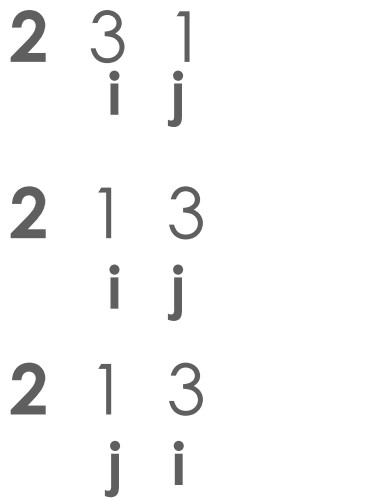




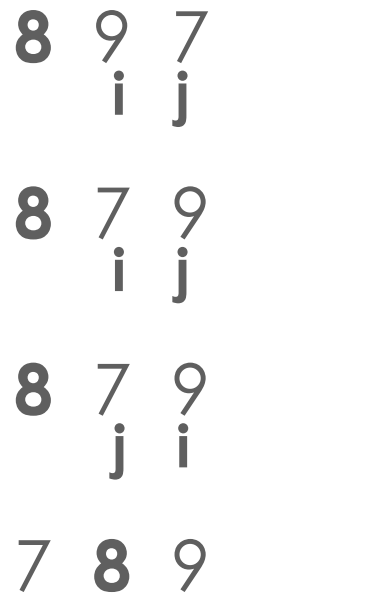
QuickSort Example



Recursive Call Quicksort (2 3 1) & Quicksort (8 9 7)

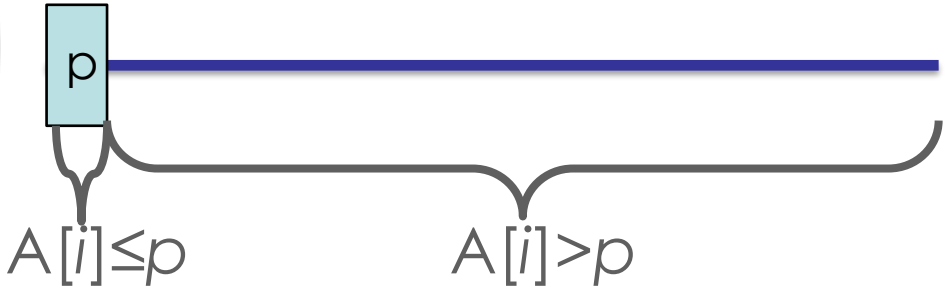


Recursive Call Quicksort (1) & Quicksort (3)



Recursive Call Quicksort (7) & Quicksort (9)

Worst-Case Efficiency of QuickSort

- In the worst case all splits are completely skewed
  - 

- For instance, an already sorted list (ascending order)!
- One subarray is empty, other reduced by only one:
  - Make *n+1* comparisons
  - Exchange pivot with itself
  - Quicksort left = Ø, right = A[1..n-1]
  - $C_{worst} = (n+1) + n + \ldots + 3 = (n+1)(n+2)/2 - 3 = \Theta(n^2)$

General Efficiency of QuickSort

- While worst case is $\Theta(n^2)$, best case (split in the middle) is $\Theta(n \log n)$ and average case (random split) is $\Theta(n \log n)$
- Improvements (in combination 20-25% faster):
  - Better pivot selection: median of three partitioning avoids worst case in sorted lists
  - Switch to Insertion Sort on small subsets
  - Elimination of recursion
- Considered the method of choice for internal sorting for large lists ($n \geq 10000$)